CS3160 Lab 9

The Cache: Mapping and Replacement

Overview

Every simulator so far has assumed a memory that responds within the cycle of the request, which no real machine provides. This lab removes that assumption.

From this lab onwards the machine is described by a configuration file, which changes the cost of a program without changing its result. The file describes a cache, to be implemented in `src/cache.cpp`.

This lab implements the *read* path: how an address is divided, where a block may be placed, how a lookup determines a hit, and which block is replaced when a set is full. The write path is the subject of Lab 10.

▶ *A cache without a write path can still serve instruction fetches, since the instruction stream is read-only. The four graded machines therefore place a small cache in front of instruction fetch only; loads and stores access the RAM directly.*

1 Configuring the machine

```
build/sim --proc=forwarding --config=configs/icache-direct.ini \
          tests/images/7-matmul.r5ob
```

Without `--config`, the memory is ideal, which remains appropriate when debugging the processor rather than the memory. `docs/the-memory-hierarchy.md` documents the file format and every reported counter; read it before starting.

The four graded configurations hold **the same 128 bytes in 16-byte blocks** and differ only in organisation:

File	Sets	Ways	Replacement
`icache-direct.ini`	8	1	FIFO (no choice with one way)
`icache-2way-lru.ini`	4	2	LRU
`icache-full-lru.ini`	1	8	LRU
`icache-full-fifo.ini`	1	8	FIFO

The size is deliberately small. Eight blocks can be traced by hand, and the test programs are small enough that a realistic cache would hold each one entirely, so that every configuration would report the same result.

2 Address decomposition



offset
selects a byte within the block: the low $\log_2(block_size)$ bits.

set
selects the set: the next $\log_2(n_sets)$ bits.

tag
is the remaining upper bits, stored with the block so that a later lookup can determine whether the block present is the one requested.

`cache_split()` performs the decomposition and `cache_block_addr()` inverts it. `log2_exact()` in `src/common.h` computes the bit counts; the configuration is checked beforehand to contain only powers of two.

▶ *A fully associative cache has one set, hence* $\log_2(1) = 0$ *set bits and a 32-bit shift. In C++ a shift by the full width is* undefined*, not zero, so this case must be handled explicitly. It is the most common error in this lab.*

3 Placement

Direct-mapped, set-associative and fully associative caches are one implementation with different numbers of blocks per set, determined solely in `cache_init()`:

`mapping`	`assoc`	`n_sets`
`MAP_DIRECT`	1	one set per block
`MAP_SET`	as configured	blocks / `assoc`
`MAP_FULL`	every block	1

No other code refers to the mapping. Decomposition, lookup and replacement read only `assoc` and `n_sets`.

4 Lookup

`cache_find(c, set, tag)` searches the ways of one set and returns the way holding the tag, or -1. The blocks of set s are `c.blocks[s * c.assoc + 0 .. c.assoc - 1]`.

A way matches only if it is **valid** *and* its tag is equal. Block records are initialised with tag 0, so a lookup that ignores `valid` reports a hit on address 0 before any block has been loaded.

This function alone distinguishes a hit from a miss: 2 cycles from 102.

5 Replacement

`c.order[set]` is a list of way numbers; **the way at the front is the next to be replaced**. Three functions maintain it:

`cache_order_insert`
A block has arrived in this way and so is the furthest from replacement: move it to the back, removing any existing entry first. A duplicate entry makes the set appear fuller than it is.

`cache_order_touch`
The way has been read. FIFO depends only on when a block *arrived*, so does nothing; LRU depends on when it was last *used*, so does as insert does.

`cache_choose_way`
Prefer an invalid way, which can be filled at no cost. Only when every way is valid does the policy apply: choose the front of the list.

`cache_order_touch()` is a single `if`, and that condition is the entire difference between FIFO and LRU; hence the two policies share one list.

6 Eviction and fill

`cache_evict_for()` clears the way selected by the policy; `cache_fill()` then reads the block from the level below and installs it.

▶ ***A miss is not necessarily an eviction.*** *The first miss into an empty set displaces nothing.* `evictions` *counts* valid *blocks removed, and so is less than* `misses` *by the number of blocks placed into free ways.*

`cache_writeback_if_dirty()`, called from `cache_evict_for()`, does nothing in this lab, since no block becomes dirty without a write path. Leave it unchanged; it becomes relevant in Lab 10.

7 The read

`Cache::read()` combines the above:

1. count the read and charge this level's latency, which applies to a hit and a miss alike;
2. decompose the address and look it up;
3. on a hit, count it and inform the replacement policy that the block was used;
4. on a miss, count it and call `cache_fill()`, which returns the way filled;
5. return `size` bytes, little-endian, from that way starting at the offset.

The bytes are read from the *block*, not from the level below: after a fill this level holds them, and reading the level below again would be both slower and incorrect.

8 Measurement

Run each program on each machine and record `I1 misses` from the summary. The expected values are:

Machine	`1-even`	`4-histogram`	`7-matmul`
`icache-roomy.ini` (4 KiB)	24	23	65
`icache-direct.ini`	25	24	95
`icache-2way-lru.ini`	25	23	97
`icache-full-lru.ini`	24	23	127
`icache-full-fifo.ini`	24	23	144

`icache-roomy.ini`, in `configs/` and not graded, is 4 KiB, larger than any of these programs, so it never evicts. Its misses are therefore the **compulsory** misses, which no organisation can avoid; they form a lower bound, and the excess over it is the cost of the organisation.

Two observations follow:

- On `1-even`, associativity helps: the fully associative cache reaches the lower bound and the direct-mapped cache does not, because two of the program's blocks conflict in one set while six sets are unused.
- On `7-matmul`, associativity *hurts*: the direct-mapped cache misses 95 times and the fully associative LRU cache 127 times.

Consider the cause before reading further. The inner loop of `7-matmul` traverses more than eight blocks and then returns to its start. In such a cycle the least recently used block is precisely the one needed next, so LRU misses on every access. The conflicts of the direct-mapped cache, by contrast, are determined by the addresses, and some accesses escape them.

▶ *This effect is not an artefact of the small cache. "Higher associativity is better" describes* average *behaviour, and a loop slightly larger than the cache is the standard counter-example.*

9 Testing

```
make          # build build/sim
make test     # build, then unit tests, then programs
make unit     # unit tests only
```

Each program test is a *program on a machine*: `icache-direct-7-matmul` is `7-matmul` on the machine described by `configs/icache-direct.ini`; three programs on four machines give twelve tests.

The tests compare *statistics*, not traces. A cache changes the cost of a program, never its result, so a simulator without a cache retires the same instructions in the same order; the trace cannot distinguish a working cache from an absent one, whereas the counters can. `instructions` and `exit_status` are also checked, because a cache that returns incorrect bytes *does* change the result.

The unit tests check each function on a few addresses chosen so that the expected result can be computed by hand. Begin with them: an incorrect decomposition fails every program without indicating the cause, but fails `split_cuts_an_address_by_the_geometry` unambiguously.

10 Submission

The folder is collected at the end of the session; **only its contents at that moment are graded**.

- `details.txt`: name, roll number, machine number.
- `src/cache.cpp`.